

Programação Concorrente (ICP-361)

Comunicação entre *threads* por memória compartilhada

Prof. Silvana Rossetto
Prof. Gabriel P. Silva
Instituto de Computação (IC - UFRJ)
Abril de 2026
gabriel@ic.ufrj.br

*Texto baseado no livro “Introdução à programação paralela em GPU para a implementação de métodos numéricos”,
D. Alfaro e S. Rossetto, Notas em Matemática Aplicada, vol. 84, 2016, disponível em:
https://www.sbmec.org.br/wp-content/uploads/2022/08/livro_84.pdf

1 Introdução

Os exemplos apresentados até agora são facilmente paralelizáveis, pois podem ser decompostos em tarefas menores quase sem troca de informação entre elas. A maioria dos problemas, contudo, exige um esforço maior de paralelização, já que há necessidade de uma troca maior de informações entre as tarefas, e a sincronização do acesso às variáveis compartilhadas e das diversas etapas de execução.

Considere o problema da soma dos elementos de um vetor A com N elementos. O programa a seguir implementa um algoritmo sequencial para resolver esse problema. A função `main()` cria uma variável local para armazenar o resultado da soma de todos os elementos do vetor (`soma_total`), chama a função que executa a soma e então imprime o resultado.

```
#define N 1000000 // Numero de elementos do vetor

//Soma os n elementos do vetor a
float somaElemVetorSeq(const float a[], int n) {
    int i;
    float soma = 0;
    for(i = 0; i < n; i++)
        soma = soma + a[i];
    return soma;
}

void main() {
    float a[N], soma_total;
    int i;
```

```

//inicializa o vetor a
...
soma_total = somaElemVetorSeq(a, N);
//imprime o valor da variavel 'soma_total'
}

```

Exemplo 1: Algoritmo sequencial para soma de vetor

O algoritmo é paralelizado dividindo-se o vetor em P blocos consecutivos de $\approx N/P$ elementos, atribuindo-se a execução de cada bloco a uma *thread* distinta. O programa seguinte apresenta essa solução.

```

#define N 1000000 //N igual ao número de elementos do vetor
#define P 4 //P igual ao número de unidades de processamento

float soma_total = 0;

//Soma uma parte dos n elementos do vetor a
void somaGrupoConsElemVetor(const float a[], int inicio, int fim) {
    int i;
    float soma_local = 0;
    for(i=inicio; i<fim; i++) {
        soma_local = soma_local + a[i];
    }
    // !!! solicita exclusão mútua !!!
    soma_total = soma_total + soma_local;
    // !!! libera a exclusão mútua !!!
}

void main() {
    float a[N];
    int i, inicio, fim;
    //inicializa o vetor a
    ...
    //soma os n elementos de a
    for(i=0; i<P; i++) {
        inicio = i * (N/P);
        //o último fluxo trata os elementos restantes
        if (i<P-1) {
            fim = inicio + (N/P);
        } else {
            fim = N;
        }
        //dispara uma thread para executar:
        // somaGrupoConsElemVetor(a, inicio, fim);
    }
    // !!! espera todos os fluxos de execução terminarem !!!
    //imprime o valor da variável 'soma_total'
    printf("soma_total=%f", soma_total);
}

```

Exemplo 2: Algoritmo paralelo para soma de vetor (com exclusão mútua)

Diferente de exemplos anteriores, onde a saída tinha a mesma dimensão da entrada, aqui o vetor de entrada produz um escalar que acumula as somas parciais de cada *thread*. A variável `soma_total` é declarada como global para que a função `somaGrupoConsElemVetor` possa alterá-la diretamente na memória. Um outra alternativa seria passar o endereço de memória dessa variável como argumento para a função a ser executada por cada *thread*.

Dentro da função `somaGrupoConsElemVetor`, cada *thread* calcula a soma dos elementos do vetor no intervalo definido e armazena essa soma em uma variável local chamada `soma_local`. Essa abordagem permite que as *threads* trabalhem de forma completamente independente enquanto somam os valores dos elementos dentro do intervalo dado.

Quando terminam de realizar essa soma, as *threads* precisam adicionar o valor da sua soma local à soma total. Porém, se dois ou mais fluxos de execução atualizarem a variável global `soma_total` ao mesmo tempo, o resultado das várias atualizações poderá ficar incorreto.

Quando uma *thread* executa `soma_total += soma_local`, ela:

1. lê `soma_total` e `soma_local` da memória para registradores;
2. soma os valores nos registradores;
3. grava o resultado de volta na memória em `soma_total`.

Se duas *threads* executarem isso simultaneamente, ambas leem o mesmo valor inicial de `soma_total` (ex: 0), fazem `0 + soma_local` e gravam seus resultados. O último a escrever na memória sobrescreve as demais atualizações, perdendo os valores anteriores.

1.1 Sincronização por exclusão mútua

Para evitar esse problema, é preciso garantir que as *threads* concorrentes executarão este trecho com exclusão mútua, isto é, apenas uma *thread* por vez. A sentença `soma_total = soma_total + soma_local` deve ser atômica, isto é, ela deve ser completamente executada antes que outra *thread* comece a processá-la. Assim, quando uma *thread* terminar sua execução, é garantido que o valor da variável global `soma_total` na memória estará atualizado. A próxima *thread* irá ler o valor atual de `soma_total` e acrescentar o valor de sua `soma_local`.

A exclusão mútua é um tipo de sincronização que pode ser necessária dentro de um algoritmo concorrente. As *threads* precisam sincronizar suas ações para evitar a ocorrência de erros de execução próprios da programação concorrente. Isso é enfatizado no programa acima, assumindo por ora que algum mecanismo de exclusão mútua (fornecido pela linguagem 'C') será invocado nessas linhas para sinalizar, respectivamente, a entrada e saída de um trecho de código atômico. Essas questões são detalhadas na próxima seção.

2 Comunicação e sincronização entre *threads* via memória compartilhada

A facilidade oferecida por um espaço de endereçamento compartilhado é normalmente usada para implementar a comunicação entre as *threads* de uma aplicação. Quando uma *thread* precisa trocar/compartilhar uma informação com outra *thread*, isso é feito alterando-se o valor de uma variável comum às duas *threads*, ou seja, uma variável que

está armazenada em um endereço de memória que as duas *threads* compartilham (conhecem e podem acessar).

O modelo de organização tipicamente adotado nos computadores multiprocessadores é baseado no uso de memória compartilhada, i.e., um espaço de endereçamento físico único é compartilhado entre todos os processadores. Esse modelo de organização facilita o desenvolvimento de aplicações para essas arquiteturas, pois é uma extensão direta do modelo de acesso à memória usado nos computadores uniprocessadores: diferentes linhas de execução (*threads*) podem acessar variáveis de um programa que estão em endereços de memória igualmente acessíveis por outras *threads*.

Quando duas ou mais *threads* de um programa precisam trocar informações entre si, o programador define variáveis de acesso compartilhado entre elas. Desse modo, quando uma *thread* tem um valor novo que deve ser comunicado para as demais *threads*, ela simplesmente escreve esse valor na variável compartilhada, independente das demais *threads* estarem esperando por esse valor naquele momento. Quando outra *thread* precisar saber qual é o valor atual dessa informação, ela simplesmente lê o conteúdo atual da variável compartilhada.

Assim, toda a comunicação entre as *threads* se dá de forma assíncrona, i.e., as *threads* escrevem/lêem valores nas variáveis compartilhadas a qualquer tempo. Para garantir que a comunicação ocorra de forma correta (por exemplo, que o valor da variável não seja sobrescrito antes que as outras *threads* leiam o valor anterior), a interação entre *threads* via memória compartilhada gera a necessidade de sincronização. A necessidade de sincronização é um dos principais desafios para a programação concorrente com memória compartilhada.

Neste texto serão discutidos aspectos relacionados com a comunicação e sincronização entre *threads* distintas dentro de uma mesma aplicação, usando o modelo de memória compartilhada.

2.1 Seções críticas e ações atômicas

Quando programas concorrentes compartilham variáveis entre *threads*, é importante que a execução das expressões aritméticas ou sentenças de atribuição da linguagem de programação sejam feitas de forma atômica, i.e., que uma vez iniciada a ação por uma *thread* ela seja executada por completo antes que outra *thread* opere sobre a mesma variável. Entretanto, as expressões aritméticas e sentenças de atribuição das linguagens de alto nível (C, Java, etc.) são normalmente traduzidas em mais de uma instrução de máquina. Por conta disso, durante a execução das sentenças ou expressões da linguagem de alto nível por *threads* diferentes, pode ocorrer entrelaçamento das instruções de máquina dessas *threads*, ocasionando resultados inesperados.

Como exemplo, considere duas *threads*, T1 e T2, associadas aos seguintes trechos de código (as variáveis y e z são inicialmente iguais a 0):

T1:	T2:
$x = y + z;$	$y = 1;$
	$z = 2;$

Se for assumido (incorretamente) que a execução de cada sentença de atribuição é uma ação atômica, o valor final de x , computado pela *thread* T1, poderá ser 0, 1 ou 3, representando as somas $0 + 0$, $1 + 0$ e $1 + 2$, respectivamente. Entretanto, as instruções de máquina para as *threads* T1 e T2 poderão ser da seguinte forma:

T1:	T2:
(1) mov y, %eax	(4) mov 1, y
(2) add z, %eax	(5) mov 2, z
(3) mov %eax, x	

Desse modo, os seguintes entrelaçamentos de instruções de máquina poderão ocorrer:

- (1), (2), (3), (4), (5) $\rightarrow x = 0$
- (4), (1), (2), (3), (5) $\rightarrow x = 1$
- (1), (4), (5), (2), (3) $\rightarrow x = 2$ (**resultado não esperado**)
- (4), (5), (1), (2), (3) $\rightarrow x = 3$

O exemplo acima ilustra que apenas as instruções de máquina são ações atômicas (e não as expressões e sentenças da linguagem de alto nível). Então, o entrelaçamento da execução das instruções de máquina é que determina o resultado final da computação. Na terceira combinação de entrelaçamento, o resultado será $x = 2$, que é um resultado não esperado. Esse tipo de erro, gerado pela execução concorrente das *threads* que acessam uma mesma área de memória, onde pelo menos uma das operações é de escrita, é chamado **corrida de dados** (do termo em inglês *data race*).

Toda referência a uma variável que pode ser acessada/modificada por outra *thread* é uma referência crítica. Para impedir a ocorrência de resultados indesejáveis para uma determinada computação, os trechos de código que contêm referências críticas devem ser executados de forma atômica. Esses trechos de código são denominados de **seção crítica** do código. Identificar os trechos de seção crítica e coordenar a sua execução é um dos problemas fundamentais da programação concorrente.

2.2 Seções de entrada e saída da seção crítica

Mecanismos de sincronização de códigos permitem transformar uma seção crítica em uma ação atômica usando dois outros trechos de código especiais chamados **seção de entrada** e **seção de saída**. Essas duas seções adicionais de código tem por finalidade “cercar” a entrada e saída da seção crítica, garantindo que os requisitos de execução atômica sejam atendidos. O pseudocódigo abaixo ilustra essa estratégia:

```
while(true) {
    executa fora da seção crítica (...)
    requisita a entrada na seção crítica //seção de entrada
    executa a seção crítica (...)
    //seção crítica
    sai da seção crítica
    //seção de saída
}
```

Exemplo 3: Pseudocódigo com seções de entrada/saída

A Figura 1 ilustra um exemplo de execução de dois processos que entram e saem das suas seções críticas (trecho de código que acessa um objeto compartilhado por duas *threads*).

A garantia de execução atômica de uma seção crítica de código consiste em escrever as seções de entrada e saída para essa seção crítica atendendo aos seguintes requisitos [1, 2]:

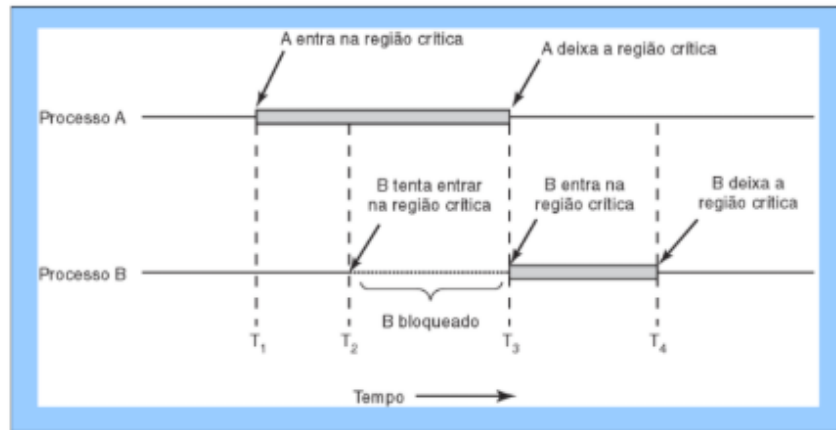


Figura 1: Exemplo de entrada e saída na seção crítica por duas *threads* (fonte: Pearson).

1. **Exclusão mútua:** quando uma *thread* está executando a sua seção crítica nenhuma outra *thread* pode executar a seção crítica relacionada com o mesmo objeto. Neste caso, as demais *threads* devem esperar na seção de entrada (se por algum motivo a *thread* na seção crítica perder o uso do processador, ela permanece com a posse da seção crítica).
2. **Ausência de inanição (*starvation*):** se uma *thread* está tentando entrar na seção crítica, então em algum momento isso deverá ser possível.
3. **Independência do restante do código:** uma *thread* que termina sua execução fora da sua seção crítica (ou entra em laço infinito fora da seção crítica) não deve interferir nas seções de entrada e saída das *threads* restantes, e quando nenhuma *thread* estiver na seção crítica e houver *threads* que desejam executá-la, apenas as *threads* que estão executando as seções de entrada e saída podem participar da decisão de qual delas ganhará o acesso à seção crítica.
4. **Garantia de entrada:** quando nenhuma *thread* está na seção crítica e uma *thread* deseja executar a seção crítica, ela deve passar pela seção de entrada imediatamente, e quando uma *thread* executa a seção de saída e há outras *threads* esperando na seção de entrada, uma delas deverá ser escolhida para entrar na seção crítica.

Para que as *threads* possam executar corretamente e chegar ao final das suas execuções, é necessário que toda *thread* que entre na seção crítica saia dela em algum momento, ou seja, uma *thread* não deve executar um laço infinito ou terminar a sua execução dentro de uma seção crítica.

Uma questão básica para o funcionamento desse tipo de mecanismo é que as seções de entrada e de saída devem ser ações atômicas (caso contrário a sincronização por exclusão mútua não será garantida).

2.3 Sincronização por exclusão mútua

A sincronização por exclusão mútua visa garantir que os trechos de código em cada *thread* que acessam objetos compartilhados não sejam executados ao mesmo tempo, ou que uma

vez iniciados sejam executados até o fim sem que outra *thread* inicie a execução do trecho equivalente. Essa restrição é necessária para lidar com a possibilidade de inconsistência dos valores das variáveis compartilhadas.

Como exemplo, considere o caso de uma operação que incrementa o valor de uma variável global (ex., `s++`). Para realizar o incremento, três instruções de nível mais baixo (linguagem de máquina) são normalmente necessárias:

- (i) ler o valor atual da variável;
- (ii) incrementar esse valor; e
- (iii) escrever o novo valor na variável global.

Se duas *threads* executam essa operação concorrentemente, as duas poderão ler o mesmo valor inicial de `s` e realizar o incremento sobre esse valor. Ao final, embora duas operações de incremento tenham sido realizadas, o valor em `s` irá refletir apenas um incremento.

A solução para a exclusão mútua é definida agrupando-se sequências contínuas de ações atômicas de *hardware* em seções críticas de *software*. As seções críticas (trechos de código que acessam objetos compartilhados) devem ser executadas atômicamente, de modo que não haja execução concorrente com outras seções críticas que acessem a mesma variável compartilhada.

2.4 Mecanismos para sincronização por exclusão mútua entre *threads*

Existem diferentes mecanismos para implementar sincronização por exclusão mútua entre *threads*. Qualquer solução proposta deve fornecer a sincronização necessária para eliminar as condições de corrida indesejáveis, sem restringir as oportunidades de paralelismo na execução da aplicação. Há duas abordagens básicas [3]:

1. sincronização por espera ocupada;
2. sincronização por escalonamento.

A sincronização por espera ocupada faz com que a *thread* fique continuamente testando o valor de uma determinada variável até que esse valor lhe permita executar a sua seção crítica com exclusividade. Para implementar esse mecanismo de sincronização é necessário dispor de instruções de máquina que permitam ler e escrever em posições da memória de forma atômica.

O principal problema da solução por espera ocupada é que ela gasta ciclos de processador enquanto espera autorização para seguir com o seu *thread* normal. A “espera ocupada” só faz sentido nos seguintes casos:

- não há nada melhor para o processador fazer enquanto espera;
- o tempo de espera é menor que o tempo requerido para a troca de contexto entre *threads*.

Os mecanismos de sincronização por escalonamento são a alternativa mais usual. Nesse caso, quando uma *thread* não consegue entrar na seção crítica ela é bloqueada e o uso do processador pode ser cedido para outra *thread*, evitando assim o desperdício de ciclos de processamento.

2.5 Sincronização por escalonamento usando locks

Um *lock* possui uma *thread* proprietária e esta relação de posse determina características particulares das operações sobre *locks*:

- Uma *thread* requisita a posse de um *lock* *L* executando a operação *L.lock()*;
- Uma *thread* que executa a operação *L.lock()* torna-se a proprietária do *lock* se nenhuma outra *threads* já possui o *lock*, caso contrário a *thread* é bloqueada;
- Uma *thread* libera sua posse sobre o *lock* executando a operação *L.unlock* (se a *thread* não possui o *lock* a operação retorna com erro);
- Uma *thread* que já possua o *lock* *L* e executa *L.lock()* novamente não é bloqueada (apenas se o *lock* for recursivo), mas deve executar *L.unlock()* o mesmo número de vezes que executou *L.lock()* antes que outra *thread* possa ganhar a posse de *L*;

O uso de *locks* para implementar exclusão mútua se dá da seguinte forma: a operação *L.lock()* implementa a entrada na seção crítica e a operação *L.unlock()* implementa a saída da seção crítica, como ilustrado abaixo:

T1:	T2:
<i>L.lock()</i> ;	<i>L.lock()</i> ;
//seção crítica	//seção crítica
<i>L.unlock()</i> ;	<i>L.unlock()</i> ;

2.5.1 Locks na biblioteca Pthreads

A biblioteca Pthreads oferece o mecanismo de sincronização por *locks* através de variáveis especiais do tipo `pthread_mutex_t`. Por definição, Pthreads implementa *locks* não-recursivos (uma *thread* não deve tentar alocar novamente um *lock* que já possui). Para tornar o *lock* recursivo é preciso mudar suas propriedades básicas. O código abaixo ilustra o uso das operações de *lock* com a biblioteca Pthreads:

```
pthread_mutex_t mutex;
pthread_mutexattr_t mutexAttr;

pthread_mutexattr_init(&mutexAttr);
pthread_mutexattr_settype(&mutexAttr, PTHREAD_MUTEX_RECURSIVE);
pthread_mutex_init(&mutex, &mutexAttr);

pthread_mutex_lock(&mutex);
//seção crítica
pthread_mutex_unlock(&mutex);
```

Exemplo 4: Exemplo de uso de locks recursivos com Pthreads

2.6 Cálculo de Primos

Um exemplo interessante é cálculo da quantidade de números primos entre 1 e um determinado valor inteiro *N*. É um programa bem simples e apresentamos no próximo exemplo

o código fonte da versão sequencial do programa. Esse programa basicamente verifica se N é divisível por algum número ímpar entre 1 e a raiz quadrada de N , sendo que na função de teste de primalidade, os números múltiplos de 2 e 3 são descartados de imediato.

```
#include <stdio.h>
#include <time.h>
#include <math.h>

int ehPrimo(long long n) {
    if (n <= 1) return 0;
    if (n == 2 || n == 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;

    long long limite = (long long)sqrt((double)n);

    for (long long i = 5; i <= limite; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }
    return 1;
}

int main() {
    long long N;
    long long total = 0;

    printf("Digite N: ");
    scanf("%lld", &N);

    for (long long i = 3; i <= N; i += 2) {
        if (ehPrimo(i))
            total++;
    }
    printf("Quantidade de primos entre 1 e %lld: %lld\n", N,
        total);
    return 0;
}
```

Exemplo 5: Código sequencial para cálculo de números primos

Na paralelização deste programa, podemos distribuir as tarefas entre os processos em lote, passando uma faixa contínua de valores para cada *thread* verificar a quantidade de números primos no intervalo recebido. Veja a seguir um exemplo completo.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
#include <math.h>

typedef struct {
    long long inicio;
```

```
    long long fim;
    long long qtd_primos;
} Faixa;

int ehPrimo(long long n) {
    if (n <= 1) return 0;
    if (n == 2 || n == 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;

    long long limite = (long long) sqrt((double)n);

    for (long long i = 5; i <= limite; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }

    return 1;
}

void *contaPrimos(void *arg) {
    Faixa *faixa = (Faixa *) arg;
    faixa->qtd_primos = 0;

    for (long long i = faixa->inicio; i <= faixa->fim; i++) {
        if (ehPrimo(i))
            faixa->qtd_primos++;
    }
    return NULL;
}

int main() {
    long long N;
    int T;

    printf("Digite N: ");
    scanf("%lld", &N);
    printf("Digite a quantidade de threads: ");
    scanf("%d", &T);

    pthread_t *threads = malloc(T * sizeof(pthread_t));
    Faixa *faixas = malloc(T * sizeof(Faixa));

    long long tamanho = (N + 1) / T;
    long long atual = 0;

    for (int i = 0; i < T; i++) {
        faixas[i].inicio = atual;
        faixas[i].fim = (i < T-1) ? atual+tamanho : N; // Garantir
        // que o último limite não ultrapasse N
        faixas[i].qtd_primos = 0;
        atual = faixas[i].fim + 1;
    }
}
```

```

        if (pthread_create(&threads[i], NULL, contaPrimos,
            &faixas[i]) != 0) {
            printf("Erro ao criar thread %d.\n", i);
            free(threads);
            free(faixas);
            return 1;
        }
    }

    long long total = 0;
    for (int i = 0; i < T; i++) {
        pthread_join(threads[i], NULL);
        total += faixas[i].qtd_primos;
    }

    printf("Quantidade de primos entre 1 e %lld: %lld\n", N,
        total);

    free(threads);
    free(faixas);
    return 0;
}

```

Exemplo 6: Código paralelo com distribuição em blocos

Este código implementa a contagem paralela de números primos de 1 até N usando *pthread*s() com particionamento estático, criando ‘ T ’ faixas de tamanho aproximado N/T . Cada thread processa sua faixa chamando *ehPrimo(i)* para cada número. No final, *pthread_join()*, soma os contadores locais na variável *total*.

As vantagens deste código são sua relativa simplicidade e o uso de contadores locais evita o uso de *mutexes*. Como detalhe, a última thread recebe sempre *fim* = N , acumulando um eventual resto de divisão.

Contudo, esta implementação tem uma forma de distribuição de trabalho que oferece alguns inconvenientes:

- A distribuição de primos ao longo do conjunto dos números inteiros $\mathbb{Z}$ não é homogênea, diminuindo a sua frequência na medida que N aumenta;
- A iteração para os números maiores leva proporcionalmente muito mais tempo que os números menores, com razão $\sqrt{N}$.

Embora essas características sejam antagônicas, normalmente o desbalanceamento aumenta nos números na faixa mais alta, pois o laço de cálculo de primalidade requer muito mais trabalho para definir se um número é primo, podendo resultar em grande desequilíbrio de carga entre os processos.

A Tabela 1 apresenta o resultado da execução deste programa com um número de *threads* variando entre 1 – execução sequencial – e seis *threads*. Podemos constatar que a eficiência é relativamente baixa, entre 0,78 e 0,67, considerando que o programa tem baixa comunicação e quase nenhum conflito no acesso a dados compartilhados. A Figura 2 consolida os resultados de *speedup* e eficiência obtidos, que expressam o desbalanceamento de carga durante a execução do programa.

Threads	Tempo (s)	Speedup	Eficiência
1 thread	34,05	—	—
2 threads	21,72	1,57	0,78
3 threads	15,17	2,24	0,75
4 threads	11,73	2,90	0,73
5 threads	10,25	3,32	0,66
6 threads	8,51	4,00	0,67

Tabela 1: Desempenho em função do número de threads

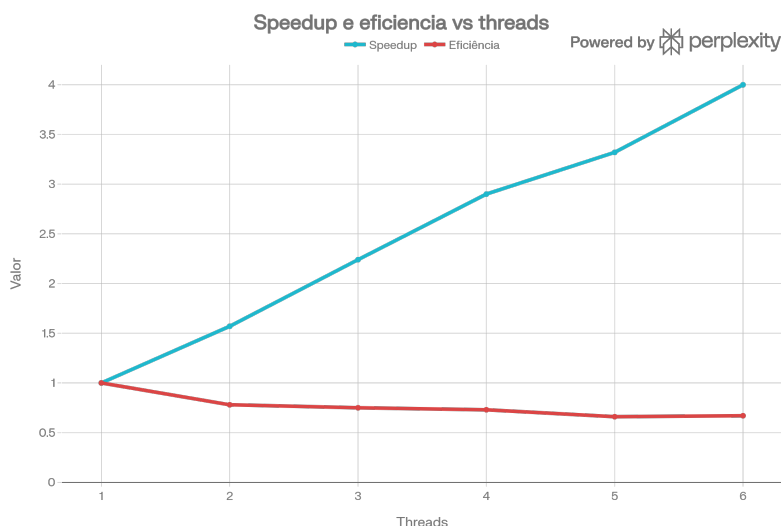


Figura 2: Speedup e Eficiência

Para superar essa limitação, adota-se o método de paralelização conhecido como saco de tarefas (*task bag*). Nesse modelo, cada *thread* busca dinamicamente um novo trabalho assim que conclui sua tarefa anterior, eliminando a rigidez das faixas fixas. Um índice global compartilhado representa o próximo número a ser testado por primalidade, acessado atomicamente por todas as *threads* através de um *mutex*. Vemos a seguir uma proposta de código que realiza esse método.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
#include <math.h>

long long N;

int ehPrimo(long long n) {
    if (n <= 1) return 0;
    if (n == 2 || n == 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;

    long long limite = (long long) sqrt((double)n);
```

```

    for (long long i = 5; i <= limite; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }
    return 1;
}

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int i_global = 3;

void *contaPrimos2(void *arg) {
    register int total = 0;
    while (1) {
        long long i_local;
        pthread_mutex_lock(&mutex);
        if (i_global >= N) {
            pthread_mutex_unlock(&mutex);
            break;
        }
        i_local = i_global;
        i_global += 2; // Incrementa de 2 para pular os números
                       pares
        pthread_mutex_unlock(&mutex);
        if (ehPrimo(i_local))
            total++;
    }
    int *ret = malloc(sizeof(int));
    *ret = total;
    pthread_exit(ret);
}

int main() {
    int T;

    printf("Digite N: ");
    scanf("%lld", &N);
    printf("Digite a quantidade de threads: ");
    scanf("%d", &T);

    pthread_t *threads = malloc(T * sizeof(pthread_t));

    for (int i = 0; i < T; i++) {
        if (pthread_create(&threads[i], NULL, contaPrimos2, NULL))
        {
            printf("Erro ao criar thread %d.\n", i);
            free(threads);
            return 1;
        }
    }

    long long total = 1; // Contando o número 2 como primo

```

```

for (int i = 0; i < T; i++) {
    void *retorno;
    pthread_join(threads[i], &retorno);
    total += *(int*) retorno;
    free(retorno);
}

printf("Quantidade de primos entre 0 e %lld: %lld\n", N,
       total);

free(threads);
pthread_mutex_destroy(&mutex);
return 0;
}

```

Exemplo 7: Código paralelo com balanceamento de carga

O funcionamento é simples e elegante: cada *thread* trava o *mutex*, pega o valor atual de ‘i_global’, testa ‘ehPrimo(i_global)’, incrementa o índice (geralmente +2 para pular pares) e libera o *mutex*. Esse ciclo se repete até ‘i_global > N’. A sincronização é mínima, ocorrendo apenas na alocação de tarefas, não no processamento.

A grande vantagem é o balanceamento de carga automático. As *threads* que terminam rápido — testando números não primos — pegam mais tarefas, enquanto *threads* “lentas”, ocupadas com regiões densas de primos, recebem menos novos números. Isso adapta-se naturalmente à irregularidade do teste de primalidade, onde a densidade de primos varia ao longo do conjunto de inteiros.

Threads	Tempo (s)	Speedup	Eficiência
1 thread	34,41	—	—
2 threads	17,79	1,93	0,97
3 threads	12,06	2,85	0,95
4 threads	9,45	3,64	0,91
5 threads	8,04	4,28	0,86
6 threads	7,19	4,79	0,80

Tabela 2: Desempenho em função do número de threads

A Tabela 2 apresenta o resultado da execução deste programa com um número de *threads* variando entre 1 – execução sequencial – e seis *threads*. Podemos constatar uma grande melhoria na eficiência, agora entre 0,95 e 0,80, mesmo considerando-se o uso de *locks* para acesso às variáveis compartilhadas entre as *threads*. A Figura 3 consolida os resultados de *speedup* e eficiência obtidos, que expressam o melhor balanceamento de carga durante a execução do programa.

Uma outra alternativa possível, deixada como exercício para o leitor, é cada processo verificar alternadamente se cada número ímpar é primo, o que na prática significa que cada processo comece verificando um número ímpar diferente ($3 + 2 \cdot \text{thread_id}$) e saltando $(\text{num_threads} \times 2)$ números entre cada número ímpar verificado.

O método de passar os números ímpares alternadamente também é preferível aos blocos de números contíguos, mas apresenta algumas irregularidades no seu desempenho, dependendo do total de *threads* que participam da computação.

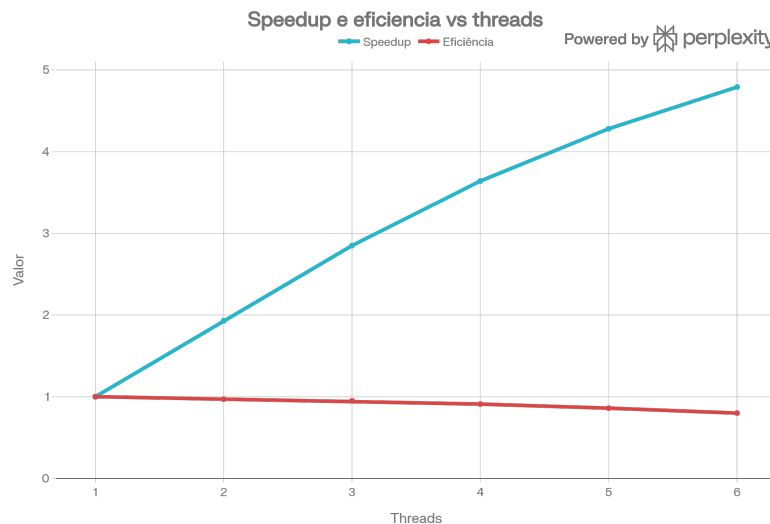


Figura 3: Speedup e Eficiência

3 Exercícios

1. Explique, com suas palavras, por que a soma de elementos de um vetor é mais fácil de paralelizar do que outros problemas que exigem comunicação frequente entre *threads*.
2. No Exemplo 2, a variável `somatotal` é compartilhada entre *threads*. Descreva por que a operação `somatotal = somatotal + somalocal` pode gerar erro se não houver exclusão mútua.
3. O texto afirma que uma operação em alto nível pode não ser atômica na prática. Explique essa ideia usando o exemplo do incremento de uma variável global.
4. Compare os mecanismos de sincronização por espera ocupada e por escalonamento. Em quais situações cada um pode ser mais adequado?.
5. Explique como um *lock* implementa exclusão mútua no acesso à seção crítica. Qual é a função de *lock* e de *unlock* nesse processo?.
6. No código de contagem de primos com divisão em blocos, explique por que a distribuição estática de trabalho pode gerar desequilíbrio entre as *threads*.
7. Analise a Tabela 1 do texto. O que os valores de *speedup* e eficiência indicam sobre o desempenho da versão com blocos fixos?.
8. Explique o funcionamento do método de saco de tarefas. Por que ele tende a melhorar o balanceamento de carga em comparação com a divisão em blocos?.
9. No Exemplo 7, o índice global `i_global` é protegido por um *mutex*. Explique por que essa proteção é necessária e qual trecho do código constitui a seção crítica.
10. O código do saco de tarefas incrementa `i_global` de 2 em 2. Explique por que essa escolha é vantajosa para a contagem de primos.

11. Compare as Tabelas 1 e 2 do texto. Por que a versão com saco de tarefas apresenta eficiência melhor, mesmo usando *mutex*?
12. Identifique nos exemplos apresentados, as seções críticas de código, se houver.
13. Em sua opinião, algum dos exemplos apresentados poderia se beneficiar do uso de espera ocupada no lugar de *locks*? Justifique.

Referências

- [1] Richard H. Carver and Kuo-Chung Tai. Modern multithreading. Wiley, 2006.
- [2] Gadi Taubenfeld. Synchronization algorithms and concurrent programming. Pearson Prentice-Hall, 2006.
- [3] M. L. Scott. Programming Language Pragmatics. Morgan-Kaufmann, 2 edition, 2006.